

LAPORAN PRAKTIKUM
POSTTEST (7)
ALGORITMA PEMROGRAMAN LANJUT



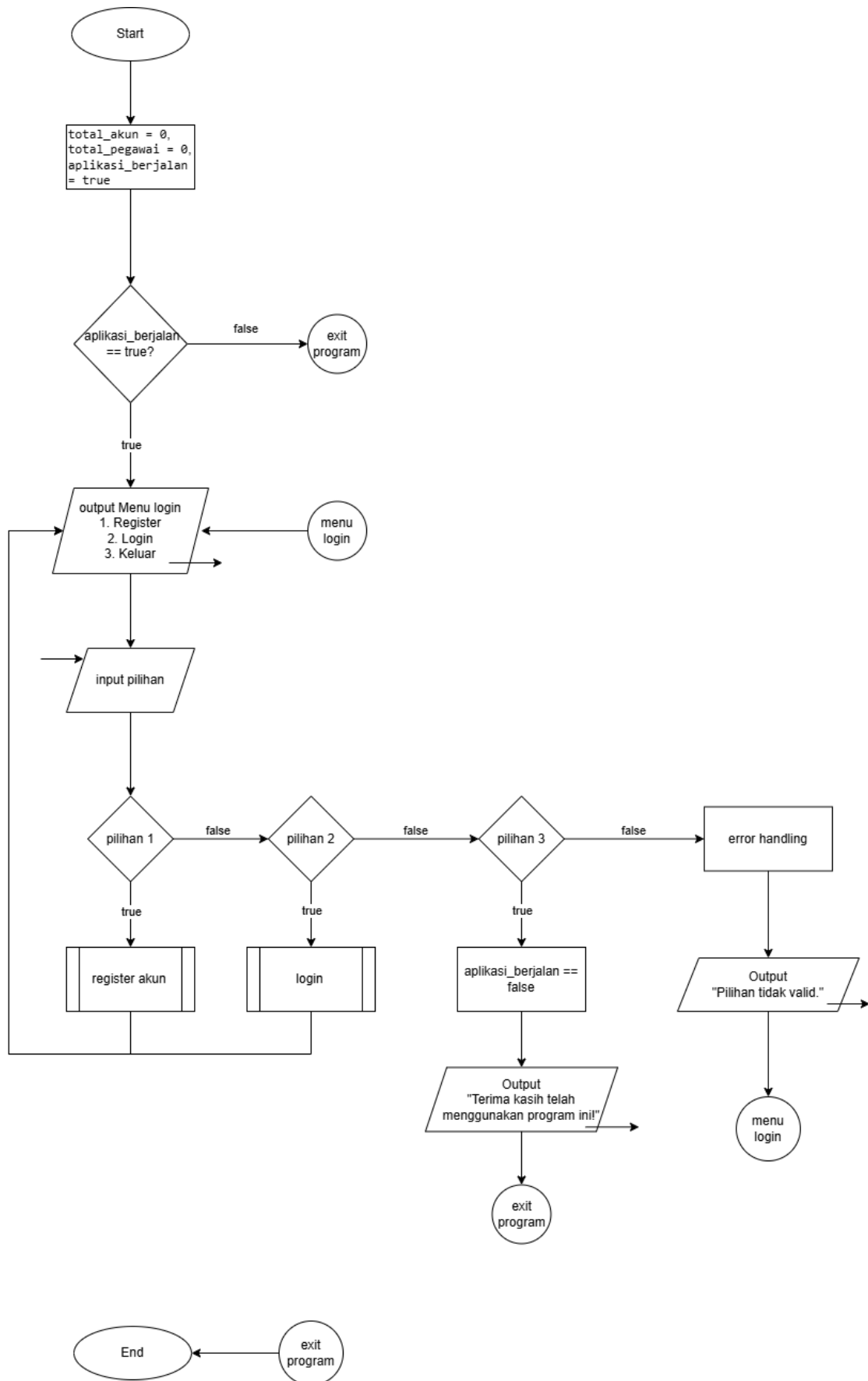
Disusun oleh:
Antung Hissyam (2509106092)

Kelas (C1 '25)

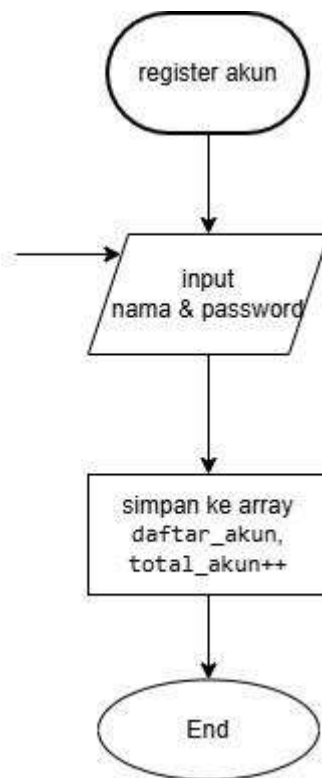
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN SAMARINDA

2026

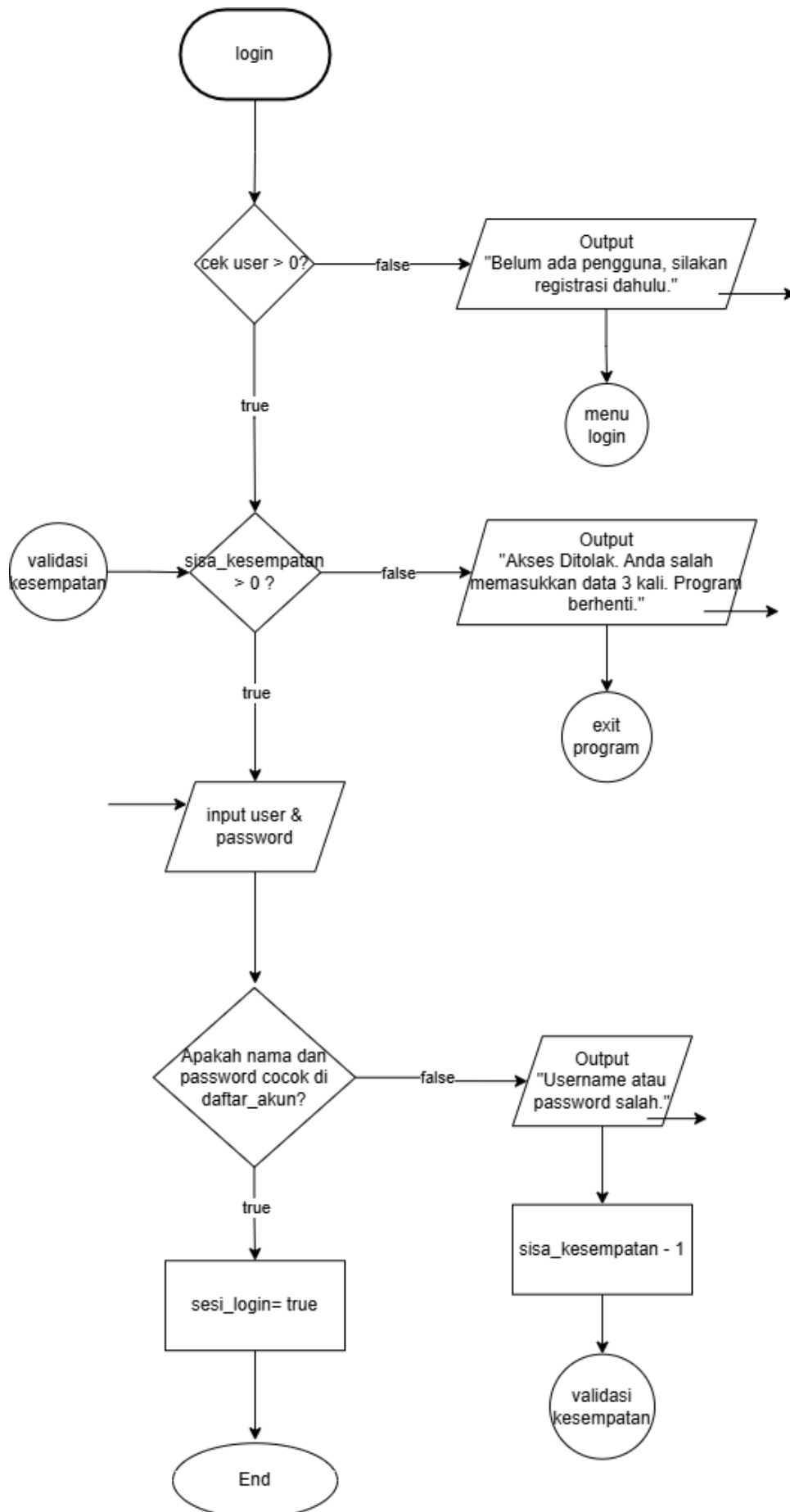
1. Flowchart



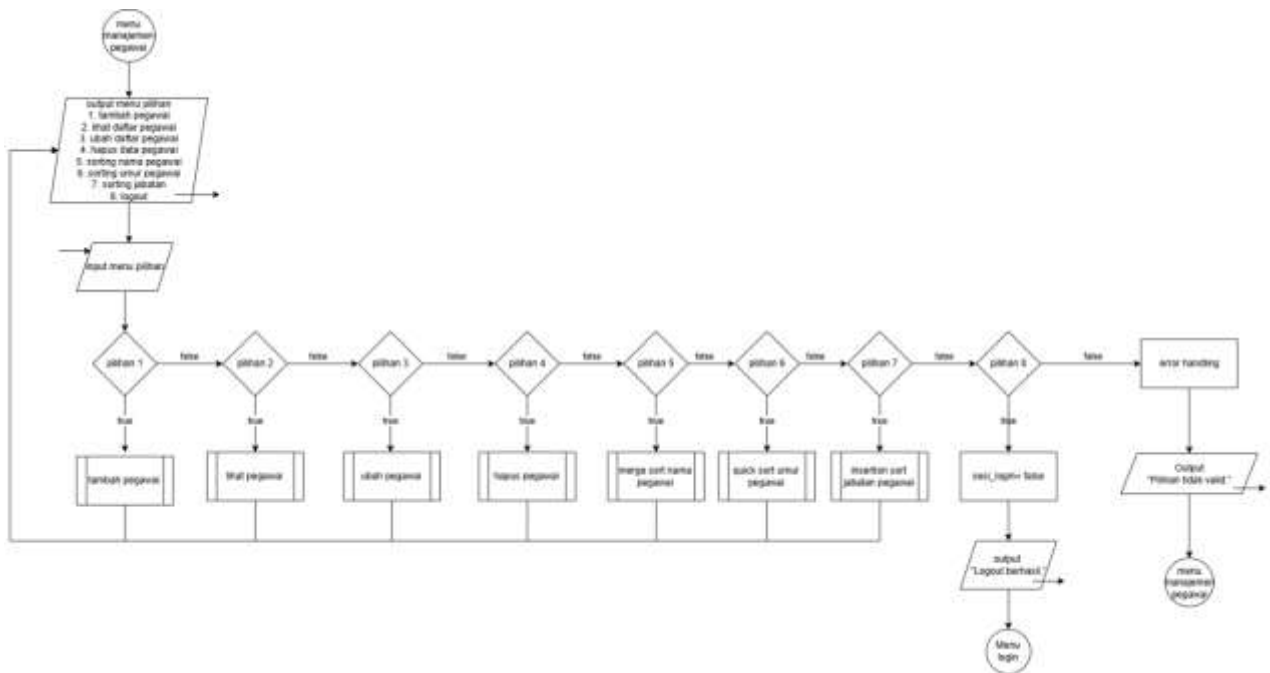
Gambar 1.1 Flowchart Menu Login.



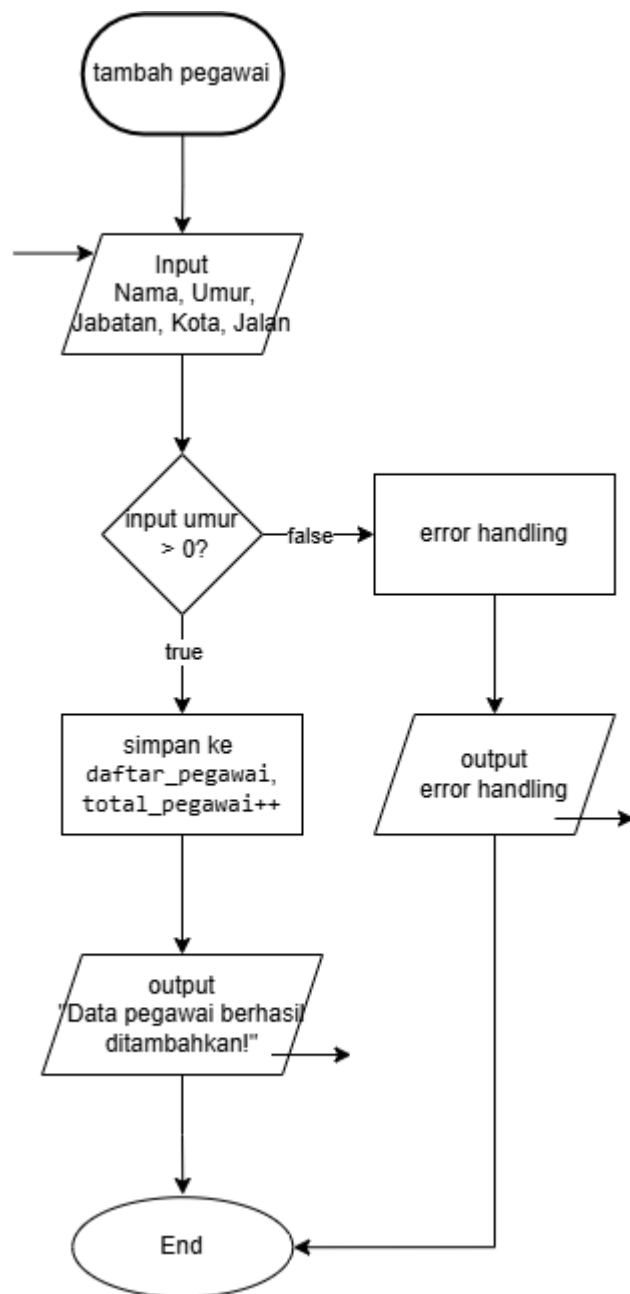
Gambar 1.2 Flowchart Fungsi Register Akun.



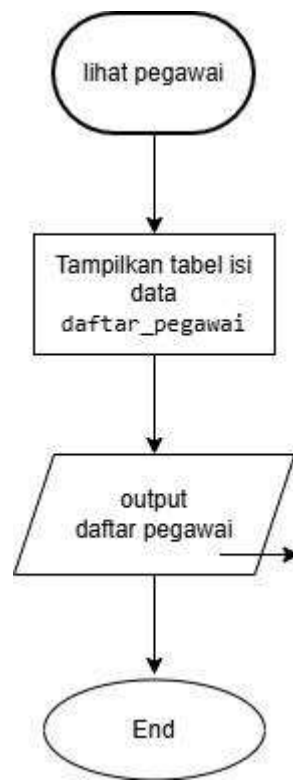
Gambar 1.3 Flowchart Fungsi Login.



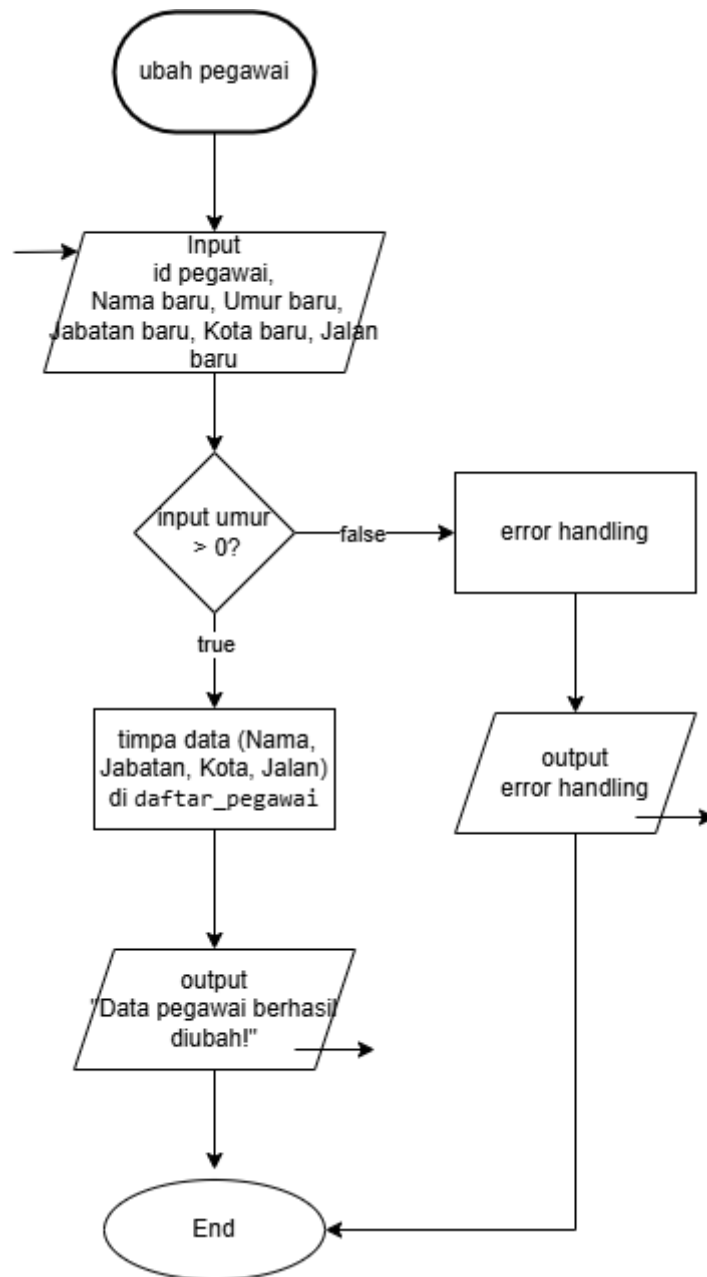
Gambar 1.4 Flowchart Menu CRUD.



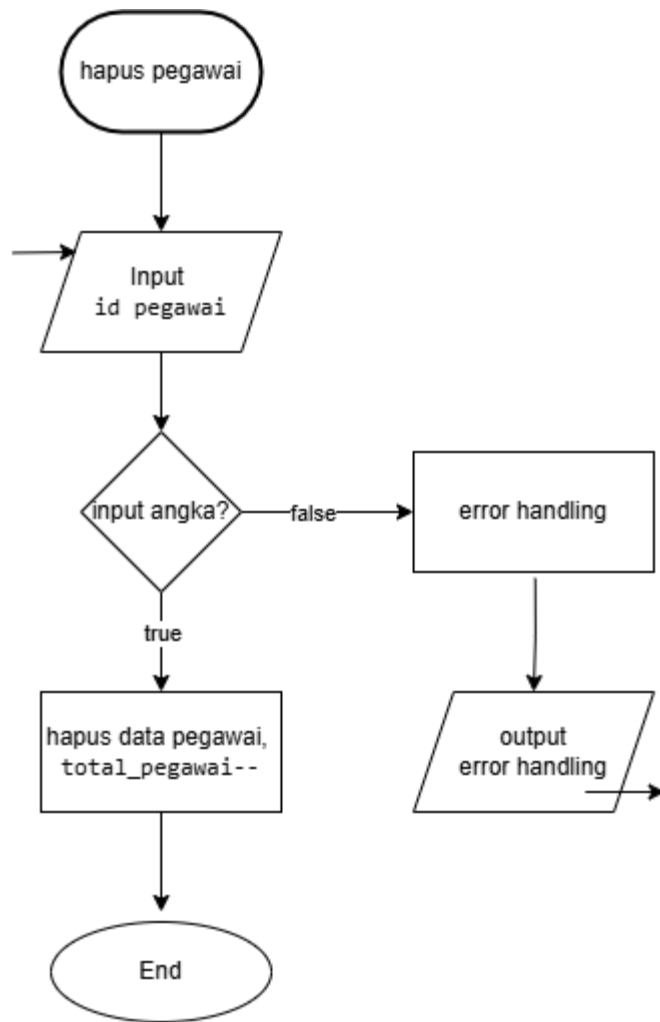
Gambar 1.5 Flowchart Fungsi Tambah Pegawai (Create).



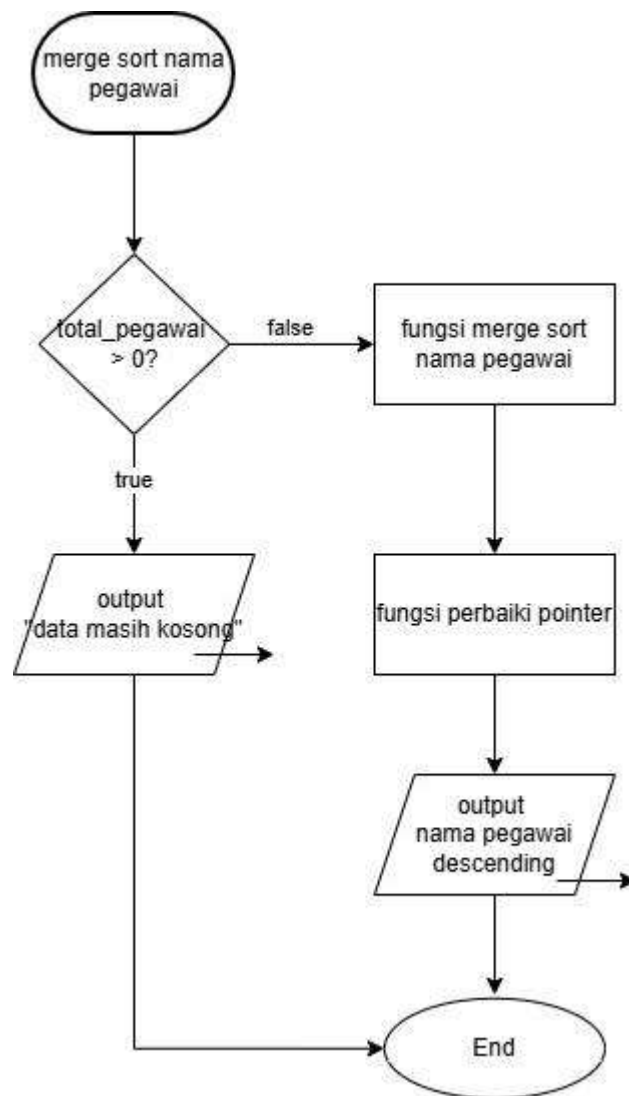
Gambar 1.6 Flowchart Fungsi Lihat Pegawai (Read).



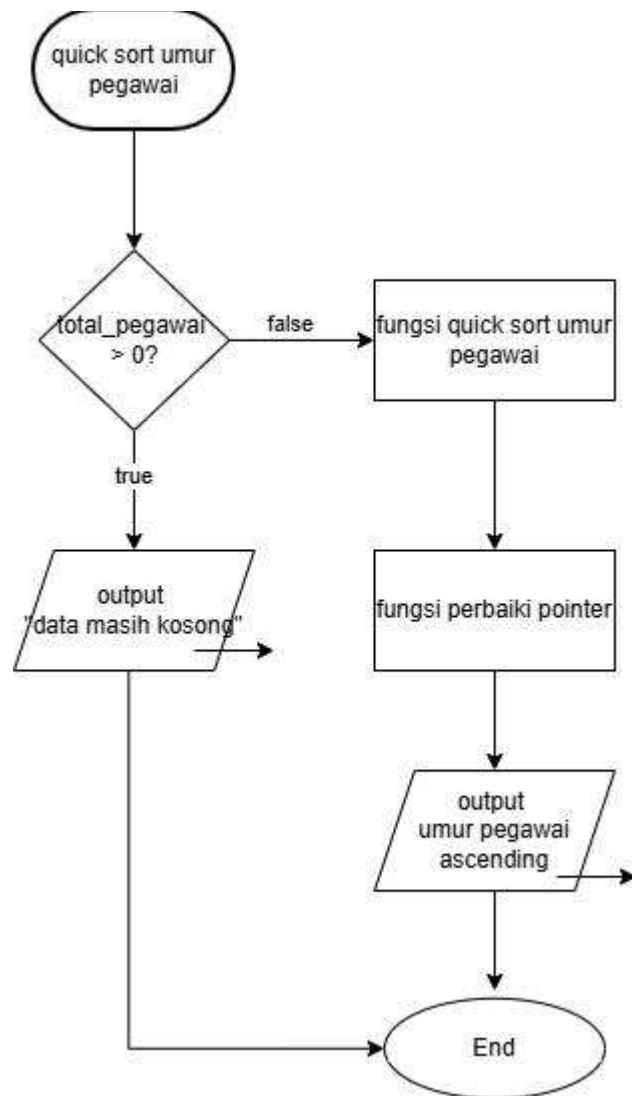
Gambar 1.7 Flowchart Fungsi Ubah Pegawai (Update).



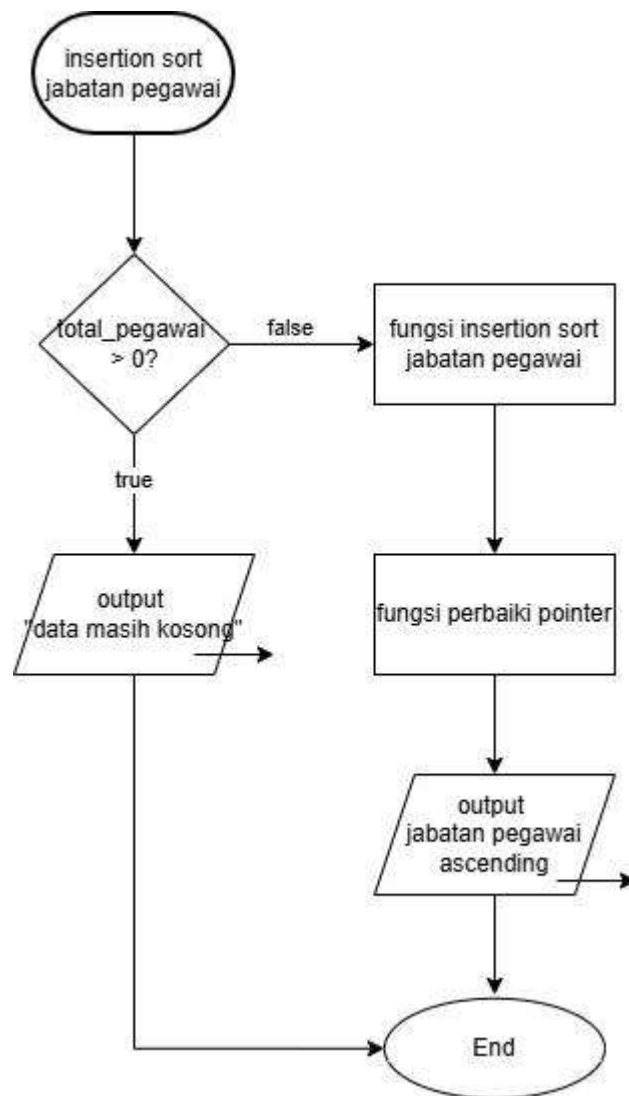
Gambar 1.8 Flowchart Fungsi Hapus Pegawai (Delete).



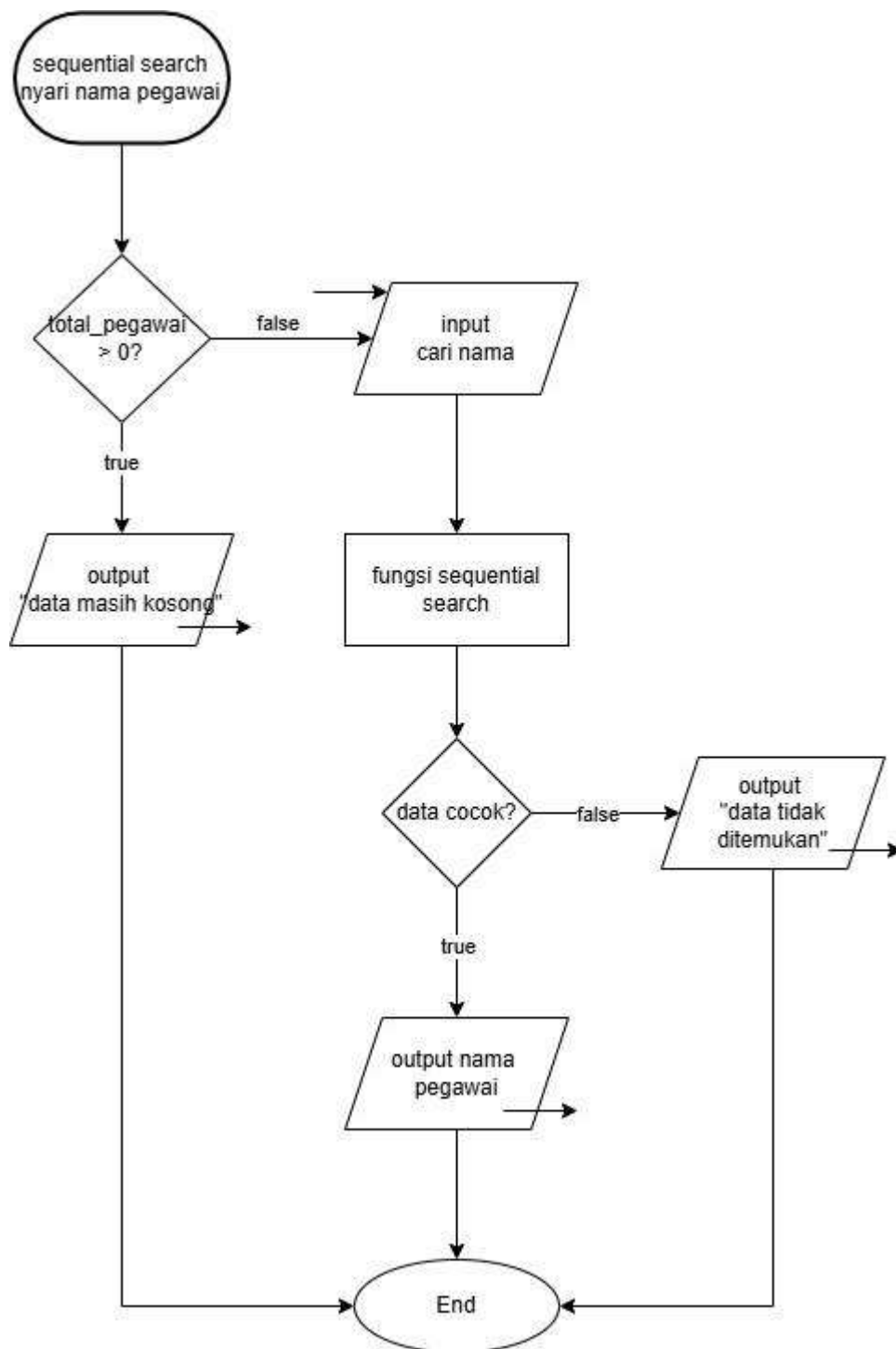
Gambar 1.9 Flowchart Fungsi Merge Sort Nama Pegawai (Descending).



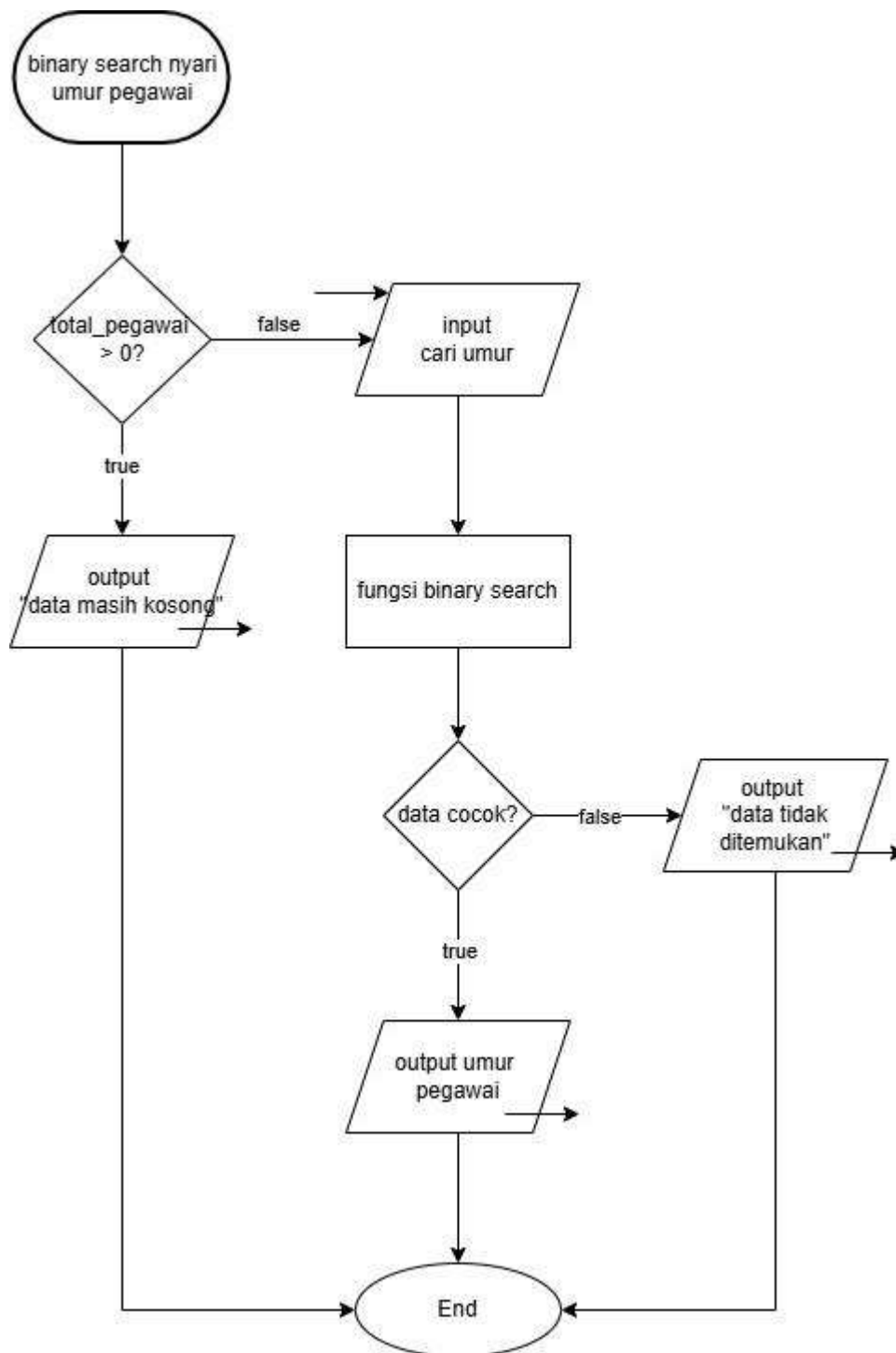
Gambar 1.10 Flowchart Fungsi Quick Sort (Ascending).



Gambar 1.11 Flowchart Fungsi Insertion Sort (Ascending).



Gambar 1.12 Flowchart Fungsi Sequential Search.



Gambar 1.13 Flowchart Fungsi Binary Search.

Penjelasan Alur Flowchart (Algoritma Program Manajemen Pegawai)

Alur logika dari Sistem Manajemen Pegawai Kebab Cendana dimulai dengan fase inialisasi variabel dan menampilkan menu autentikasi. Mulai dari menu awal ini, sistem telah dilengkapi dengan proteksi error handling, sehingga apabila pengguna memasukkan tipe data yang tidak sesuai (misalnya memasukkan huruf saat diminta memilih angka menu), program akan

menangkap kesalahan tersebut, membersihkan memori buffer, dan mengulang permintaan input. Jika pengguna memilih opsi pendaftaran, sistem akan meminta input nama beserta kata sandi untuk disimpan ke dalam struktur data, kemudian jumlah akun akan bertambah dan alur kembali ke menu awal. Jika pengguna memilih opsi masuk, mereka harus memasukkan kredensial untuk dicocokkan dengan data di memori. Pengguna diberikan batas maksimal tiga kali percobaan gagal sebelum program dipaksa berhenti, namun jika berhasil, alur akan langsung diarahkan masuk ke dalam menu manajemen pegawai yang berada di dalam sebuah perulangan.

Setelah berada di dalam menu utama, pengguna dapat melakukan operasi manipulasi data dasar. Pada tahapan ini, setiap form yang membutuhkan inputan numerik seperti umur dan nomor indeks pegawai akan dieksekusi di dalam blok pelindung try-catch. Jika pengguna memasukkan huruf, angka nol, nilai minus, atau nomor indeks yang di luar rentang tabel, sistem akan melemparkan eksepsi (throw error) dan membatalkan proses agar program tidak crash. Jika input divalidasi benar dan terjadi penambahan data baru, alamat memori dari variabel jabatan akan diikat ke dalam pointer. Khusus untuk operasi pengubahan atau penghapusan data, program akan melakukan pergeseran elemen dan langsung memanggil fungsi perbaikan pointer agar referensi alamat memori tidak rusak atau salah menunjuk akibat pergeseran tersebut.

Selain operasi dasar, pengguna juga dapat memilih menu pengurutan data. Saat menu ini dipilih, program akan memastikan terlebih dahulu bahwa data pegawai tidak kosong. Jika valid, program akan mengeksekusi algoritma Merge Sort untuk mengurutkan nama secara menurun, Quick Sort untuk mengurutkan umur secara menaik, atau Insertion Sort untuk mengurutkan jabatan secara menaik. Setelah proses pengurutan selesai dan letak elemen berpindah posisi, program diwajibkan memanggil ulang fungsi perbaikan pointer sebelum akhirnya menampilkan tabel data terbaru.

Fase penting berikutnya adalah fitur pencarian data. Untuk pencarian teks berdasarkan nama pegawai, program menggunakan algoritma Sequential Search yang melakukan perulangan secara linear dari indeks pertama hingga terakhir. Jika string yang diinputkan pengguna cocok dengan data di memori, rincian pegawai akan dicetak, dan jika tidak ada yang cocok hingga akhir pencarian, program akan mencetak pesan bahwa data tidak ditemukan. Sementara itu, untuk pencarian angka berdasarkan umur pegawai, program menggunakan algoritma Binary Search. Sama seperti operasi dasar, penginputan target umur pada fitur ini juga harus melewati validasi error handling untuk memblokir inputan bertipe huruf. Karena pencarian biner mewajibkan data dalam kondisi terurut, sistem akan memanggil fungsi pengurutan Quick Sort secara otomatis

sebelum membagi rentang pencarian untuk mencari titik tengah. Jika umur yang dicari cocok dengan titik tengah, program akan memindai elemen di sebelah kiri dan kanan titik tersebut untuk memastikan semua pegawai dengan umur kembar ikut ditampilkan ke layar.

Fase terakhir adalah penyelesaian ketika pengguna mengakhiri sesi. Jika pengguna memilih opsi keluar dari menu utama, penanda sesi akan diubah nilainya sehingga perulangan menu terhenti dan layar kembali menampilkan menu autentikasi awal. Terakhir, jika pengguna memilih opsi keluar dari aplikasi, program akan mengeksekusi fungsi rekursif untuk melakukan proses hitung mundur sebelum akhirnya program ditutup sepenuhnya.

2. Deskripsi Singkat Program

Program aplikasi Manajemen Pegawai Kebab Cendana pada Post-Test 7 ini merupakan pengembangan yang berfokus pada penerapan Error Handling dan modularitas kode. Modifikasi utama yang dilakukan meliputi tiga aspek: penanganan eksepsi, pembuatan pustaka kustom (user-defined library), dan peningkatan antarmuka (UI).

Pertama, untuk memenuhi kriteria poin plus, dilakukan restrukturisasi kode dengan membuat user-defined library berupa file header bernama `kebab.h`. File ini berfungsi sebagai wadah untuk menyimpan seluruh deklarasi struktur data (`struct Akun`, `Alamat`, dan `Pegawai`) beserta fungsi pembantu (perbaiki `Pointer`). Pemisahan ini dilakukan agar source code utama menjadi lebih bersih, modular, dan fokus pada logika operasional (`CRUD`, `Sorting`, `Searching`) tanpa dipenuhi oleh deklarasi tipe data di bagian atas program. File pustaka ini kemudian dipanggil di program utama menggunakan direktif `#include "kebab.h"`.

Kedua, program mengimplementasikan Exception Handling menggunakan blok `try`, `throw`, dan `catch`. Fitur ini ditambahkan pada bagian input pengguna (seperti pemilihan menu, input umur, dan input nomor urut pegawai) untuk mencegah program berhenti secara paksa (`crash`) apabila pengguna salah memasukkan tipe data, misalnya memasukkan huruf pada variabel bertipe integer. Jika terjadi kesalahan, program akan melempar sinyal error dan membersihkan buffer input, lalu kembali meminta input yang benar.

Ketiga, program menggunakan library eksternal `<iomanip>` dan `<stdlib.h>`. Penggunaan `<iomanip>` dimanfaatkan untuk merapikan tata letak (formatting) tabel daftar pegawai menggunakan presisi spasi otomatis (`setw` dan `setfill`), sedangkan `<stdlib.h>` digunakan untuk memanggil fungsi `system("cls")` guna membersihkan layar terminal setiap kali terjadi perpindahan menu, sehingga antarmuka program menjadi jauh lebih rapi dan interaktif.

3. Source Code

A. Implementasi User-Defined Library (kebab.h)

Pada pembaruan kali ini, program menerapkan modularitas dengan memisahkan deklarasi struktur data (Akun, Alamat, Pegawai) dan fungsi modifikasi memori ke dalam file pustaka kustom bernama `kebab.h`. Hal ini membuat source code utama menjadi lebih ringkas. Pustaka ini kemudian dipanggil ke dalam program utama menggunakan sintaks `#include "kebab.h"`.

```
#ifndef KEBAB_H
#define KEBAB_H
#include <string>
using namespace std;

struct Alamat {
    string kota;
    string jalan;
};

struct Pegawai {
    string nama_pegawai;
    int umur;
    string jabatan;
    Alamat lokasi;
    string *jabatan_ptr;
};

void perbaikiPointer(Pegawai daftar_pegawai[], int n) {
    for (int i = 0; i < n; i++) {
        daftar_pegawai[i].jabatan_ptr = &daftar_pegawai[i].jabatan;
    }
}
#endif
```

B. Implementasi Exception Handling (Try, Throw, Catch)

Untuk mencegah berhentinya program secara paksa akibat kelalaian input (human error), sistem diproteksi menggunakan metode Exception Handling. Potongan kode di bawah ini diambil dari fungsi tambahPegawai, di mana program akan mencoba membaca input umur di dalam blok try. Jika pengguna memasukkan huruf atau angka minus, program akan membatalkan input dengan throw dan melempar pesan error ke dalam blok catch untuk membersihkan buffer memori.

```
try {
    cout << "Umur          : ";
    if (!(cin >> daftar_pegawai[indeks].umur)) {
        throw runtime_error("Input umur harus berupa angka bulat!");
    }
    if (daftar_pegawai[indeks].umur <= 0) {
        throw invalid_argument("Umur tidak valid (tidak boleh nol atau minus)!");
    }
} catch (const exception& e) {
    cout << "\n[!] Gagal menambahkan pegawai: " << e.what() << "\n";
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
```

C. Penggunaan Library Eksternal

Sebagai poin plus, antarmuka program dirancang lebih interaktif dan rapi menggunakan pustaka eksternal `<iomanip>` dan `<stdlib.h>`. Pustaka `<stdlib.h>` digunakan untuk memanggil `system("cls")` yang membersihkan layar pada setiap perpindahan menu. Sementara itu, `<iomanip>` digunakan pada fungsi lihatPegawai untuk menciptakan jarak spasi presisi (`setw`) dan garis batas otomatis (`setfill`) sehingga data array dapat dicetak dalam bentuk tabel yang simetris.

```
// nampilkan header tabel yang simetris menggunakan iomanip
cout << setfill('-') << setw(85) << "-" << endl;
cout << setfill(' ');
cout << left << setw(5) << "No."
    << left << setw(20) << "Nama"
    << left << setw(8) << "Umur"
    << left << setw(20) << "Jabatan"
    << "Alamat (Kota, Jalan)\n";
cout << setfill('-') << setw(85) << "-" << endl;
```

4. Hasil Output

A. Output Menu Register dan Login Berhasil

Tampilan ini menunjukkan simulasi saat pengguna pertama kali mendaftarkan akun, kemudian berhasil melakukan login menggunakan akun tersebut.

```
=====
      Sistem Manajemen Pegawai Kebab Cendana
=====
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 1

--- Register ---
Masukkan Nama (Tanpa Spasi) : antung
Masukkan Password           : 092
[+] Akun berhasil didaftarkan! Silakan Login.

=====
      Sistem Manajemen Pegawai Kebab Cendana
=====
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 2

--- Login ---
Masukkan Nama      : antung
Masukkan Password  : 092

[+] Login Berhasil! Selamat datang, antung.

=====
      Menu Manajemen Pegawai
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Sorting Nama Pegawai (Z-A) - Merge
6. Sorting Umur Pegawai (Kecil-Besar) - Quick
7. Sorting Jabatan (A-Z) - Insertion
8. Cari Nama Pegawai (Sequential Search)
9. Cari Umur Pegawai (Binary Search)
10. Logout
Pilihan: █
```

Gambar 4.1 Output Register dan Login Berhasil

B. Output Gagal Login 3 Kali (Akses Ditolak)

Tampilan ini membuktikan bahwa fitur pembatasan akses berjalan dengan baik. Jika pengguna salah menginput data sebanyak 3 kali, program otomatis dihentikan.

```
=====
Sistem Manajemen Pegawai Kebab Cendana
=====
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 2

--- Login ---
Masukkan Nama      : aw
Masukkan Password  : aw
[!] Nama atau Password salah! Sisa: 2

Masukkan Nama      : aw
Masukkan Password  : aw
[!] Nama atau Password salah! Sisa: 1

Masukkan Nama      : aw
Masukkan Password  : aw

[X] Gagal login 3 kali. Kembali ke menu utama.
```

Gambar 4.2 Output Login Gagal dan Program Berhenti

C. Output Tambah Pegawai (Create)

Tampilan ini menunjukkan ketika pengguna (admin) berhasil masuk ke menu CRUD dan melakukan penambahan data pegawai beserta alamatnya (Nested Struct).

```
=====
Menu Manajemen Pegawai
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Sorting Nama Pegawai (Z-A) - Merge
6. Sorting Umur Pegawai (Kecil-Besar) - Quick
7. Sorting Jabatan (A-Z) - Insertion
8. Cari Nama Pegawai (Sequential Search)
9. Cari Umur Pegawai (Binary Search)
10. Logout
Pilihan: 1

--- Tambah Pegawai ---
Nama Pegawai : antung
Umur          : 20
Jabatan       : ceo
Kota          : smd
Jalan         : cendana
[+] Data pegawai berhasil ditambahkan!
```

Gambar 4.3 Output Tambah Data Pegawai

D. Output Tampilkan Data (Read)

Tampilan ini membuktikan bahwa data yang diinputkan sebelumnya berhasil disimpan di dalam array of struct dan ditampilkan dengan format tabel yang rapi.

```
=====
Menu Manajemen Pegawai
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Sorting Nama Pegawai (Z-A) - Merge
6. Sorting Umur Pegawai (Kecil-Besar) - Quick
7. Sorting Jabatan (A-Z) - Insertion
8. Cari Nama Pegawai (Sequential Search)
9. Cari Umur Pegawai (Binary Search)
10. Logout
Pilihan: 2

--- Daftar Pegawai Kebab Cendana ---
-----
No. | Nama           | Umur | Jabatan           | Alamat (Kota, Jalan)
-----
1  | antung         | 20   | ceo               | smd, cendana
-----
```

Gambar 4.4 Output Tabel Daftar Pegawai

E. Output Ubah dan Hapus Data (Update & Delete)

Tampilan ini menunjukkan proses pencarian data berdasarkan nomor indeks. Data pegawai akan diubah jika memilih opsi 3, atau dihapus jika memilih opsi 4.

```
--- Ubah Data Pegawai ---
Masukkan nomor pegawai yang akan diubah: 2
Masukkan nama baru   : nasihuy
Masukkan umur baru   : 30
Masukkan jabatan baru: ketua lapas
Masukkan kota baru   : gatau
Masukkan jalan baru  : gatau
[+] Data berhasil diubah.

=====
Menu Manajemen Pegawai
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Sorting Nama Pegawai (Z-A) - Merge
6. Sorting Umur Pegawai (Kecil-Besar) - Quick
7. Sorting Jabatan (A-Z) - Insertion
8. Cari Nama Pegawai (Sequential Search)
9. Cari Umur Pegawai (Binary Search)
10. Logout
Pilihan: 4

--- Hapus Data Pegawai ---
Masukkan nomor pegawai yang akan dihapus: 2
[+] Pegawai berhasil dihapus.
```

Gambar 4.5 Output Ubah dan Hapus Data

F. Output Pilihan Tidak Valid dan Keluar

Tampilan ini menunjukkan ketika pengguna menginput angka di luar daftar menu (contoh:

angka 6), program akan menampilkan peringatan dan mengulang menu.

```
=====
MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 6
[!] Pilihan tidak valid!
```

Gambar 4.6 Output Validasi Menu

G. Output Exit Program

Tampilan ini menunjukkan ketika pengguna memilih keluar dari program, program akan berhenti.

```
=====
SISTEM PEGAWAI KEBAB CENDANA (ADMIN)
=====
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 3
Aplikasi akan tertutup dalam 3...
Aplikasi akan tertutup dalam 2...
Aplikasi akan tertutup dalam 1...
Terima kasih telah menggunakan aplikasi ini!
```

Gambar 4.7 Output Program Selesai

H. Output Merge Sort Nama Pegawai

Tampilan ini menunjukkan ketika pengguna memilih Sorting Nama Pegawai

```
=====
MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Sorting Nama Pegawai (Z-A) - Merge
6. Sorting Umur Pegawai (Kecil-Besar) - Quick
7. Sorting Jabatan (A-Z) - Insertion
8. Logout
Pilihan: 5

[+] Data berhasil diurutkan berdasarkan Nama (Z-A).

--- DAFTAR PEGAWAI KEBAB CENDANA ---
-----
No. | Nama                | Umur | Jabatan                | Alamat (Kota, Jalan)
-----
1   | hissyam              | 19   | balikpapan             | mt haryono, cendana
2   | antung               | 20   | ceo                    | samarinda, cendana
-----
```

Gambar 4.8 Output Merge Sort (Descending).

I. Output Quick Sort Umur Pegawai

Tampilan ini menunjukkan ketika pengguna memilih Sorting Umur Pegawai

```
=====
      MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Sorting Nama Pegawai (Z-A) - Merge
6. Sorting Umur Pegawai (Kecil-Besar) - Quick
7. Sorting Jabatan (A-Z) - Insertion
8. Logout
Pilihan: 6

[+] Data berhasil diurutkan berdasarkan Umur (Muda-Tua).

--- DAFTAR PEGAWAI KEBAB CENDANA ---
-----
No. | Nama                | Umur | Jabatan                | Alamat (Kota, Jalan)
-----
1   | hissyam              | 19   | balikpapan             | mt haryono, cendana
2   | antung               | 20   | ceo                    | samarinda, cendana
-----
```

Gambar 4.9 Output Quick Sort (Ascending).

J. Output Insertion Sort Jabatan Pegawai

Tampilan ini menunjukkan ketika pengguna memilih Sorting Jabatan Pegawai

```
=====
      MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Sorting Nama Pegawai (Z-A) - Merge
6. Sorting Umur Pegawai (Kecil-Besar) - Quick
7. Sorting Jabatan (A-Z) - Insertion
8. Logout
Pilihan: 7

[+] Data berhasil diurutkan berdasarkan Jabatan (A-Z).

--- DAFTAR PEGAWAI KEBAB CENDANA ---
-----
No. | Nama                | Umur | Jabatan                | Alamat (Kota, Jalan)
-----
1   | hissyam              | 19   | balikpapan             | mt haryono, cendana
2   | antung               | 20   | ceo                    | samarinda, cendana
-----
```

Gambar 4.10 Output Insertion Sort (Ascending).

K. Output Sequential Search Nama Pegawai

Tampilan ini menunjukkan ketika pengguna memilih Searching Nama Pegawai

```
=====
Menu Manajemen Pegawai
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Sorting Nama Pegawai (Z-A) - Merge
6. Sorting Umur Pegawai (Kecil-Besar) - Quick
7. Sorting Jabatan (A-Z) - Insertion
8. Cari Nama Pegawai (Sequential Search)
9. Cari Umur Pegawai (Binary Search)
10. Logout
Pilihan: 8
Masukkan nama pegawai yang dicari: antung

--- Hasil Pencarian Nama ---
Ditemukan pada indeks 0:
Nama      : antung
Umur      : 20
Jabatan   : ceo
Alamat    : smd
```

Gambar 4.11 Output Sequential Search

L. Output Binary Search Umur Pegawai.

Tampilan ini menunjukkan ketika pengguna memilih Searching Umur Pegawai

```
=====
Menu Manajemen Pegawai
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Sorting Nama Pegawai (Z-A) - Merge
6. Sorting Umur Pegawai (Kecil-Besar) - Quick
7. Sorting Jabatan (A-Z) - Insertion
8. Cari Nama Pegawai (Sequential Search)
9. Cari Umur Pegawai (Binary Search)
10. Logout
Pilihan: 9
Masukkan umur pegawai yang dicari: 20

--- Hasil Pencarian Umur ---
Ditemukan:
Nama      : antung
Umur      : 20
Jabatan   : ceo
```

Gambar 4.12 Output Binary Search

M. Output Error Handling

Tampilan ini menunjukkan ketika pengguna memilih Menu yang tidak ada.

```
--- Sistem Manajemen Pegawai Kebab Cendana ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 4
[!] Pilihan tidak valid!
--- Sistem Manajemen Pegawai Kebab Cendana ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: █
```

Gambar 4.13 Output Error Handling 1

Tampilan ini menunjukkan ketika pengguna menginput umur yang tidak sesuai ketentuan.

```
Nama Pegawai : antung
Umur          : -999

[!] Gagal menambahkan pegawai: Umur tidak valid (tidak boleh nol atau minus)!
```

Gambar 4.14 Output Error Handling 2

5. Langkah-langkah GIT

5.1 GIT Add

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git add .
```

Perintah `git add .` berfungsi untuk menambahkan semua file yang baru dibuat atau file yang mengalami modifikasi ke dalam *staging area*.

5.2 GIT Commit

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git commit -m "PT-1"
[main f5782f9] PT-1
2 files changed, 105 insertions(+)
create mode 100644 post-test/post-test-apl-1/2509106092-AntungHissyam-PT-1.cpp
create mode 100644 post-test/post-test-apl-1/2509106092-AntungHissyam-PT-1.exe
```

Perintah `git commit` berfungsi untuk merekam dan menyimpan secara permanen perubahan file yang sudah berada di *staging area* ke dalam riwayat repositori lokal.

5.3 GIT Push

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 448.22 KiB | 6.90 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

Perintah git push berfungsi untuk mengunggah semua *commit* atau perubahan yang ada di repositori lokal (komputer) menuju ke repositori *remote* yang ada di Github.